Katzenpost thin client

This document describes the design of the new Katzenpost mixnet *thin client*. In particular we discuss its multiplexing and privilege-separation design elements as well as the protocol used by the thin-client library.

Introduction

A Katzenpost mixnet client has several responsibilities at a minimum:

- composing Sphinx packets
- decrypting SURB replies
- sending and receiving Noise protocol messages
- keeping up to date with the latest PKI document

Therefore applications will be integrated with Katzenpost using the connector library known as a thin client library which gives them the capability to talk with the connector daemon and in that way interact with the mix network. The library itself does not do any mixnet-related cryptography since that is already handled by the connector daemon. In particular, the PKI document is stripped by the daemon before it's passed on to clients using the connector library. Likewise, the library doesn't decrypt SURB replies or compose Sphinx packets, with Noise, Sphinx, and PKI related cryptography being handled by the daemon.

Connector library and daemon protocol

The thin-client daemon protocol uses a local network socket, either a Unix domain socket or TCP.

Protocol messages

Note that there are two protocol message types and they are always CBOR encoded. We send over length prefixed CBOR blobs. That is to say, a length prefix encoded as a big endian unsigned four byte integer (uint32).

The client sends the `Request` messages and the daemon sends the `Response` messages.

Protocol description

Upon connecting to the daemon socket the client must wait for two messages. The first message received must have its `is_status` field set to **true** and its `is_connected` field indicating whether or not the daemon has a mixnet PQ Noise protocol connection to an entry node.

The client then awaits the second message, which contains the PKI document in its `payload` field. This marks the end of the initial connection sequence. Note that this PKI document is stripped of all cryptographic signatures.

In the next protocol phase, the client may send `Request` messages to the daemon in order to cause the daemon to encapsulate the given payload in a Sphinx packet and send it to the gateway node. Likewise the daemon my send the client `Response` messages at any time during this protocol phase. These `Response` messages may indicate a connection status change, a new PKI document, or a message-sent or reply event.